

Fluxo Completo de Interação do Bot FitJourneyAI

1. Introdução

Este documento descreve, de forma detalhada, o fluxo completo de interação do bot FitJourneyAI, desenvolvido para auxiliar praticantes de musculação no acompanhamento da evolução física, registro de treinos e geração de planos personalizados com apoio de inteligência artificial. O objetivo é servir como referência técnica e funcional para o TCC, facilitando tanto a implementação quanto a explicação da solução.

2. Visão Geral do FitJourneyAI

O FitJourneyAI funciona como um “coach digital” integrado ao Telegram. O usuário se comunica por mensagens de texto com o bot, que responde com:

- Perguntas guiadas para cadastro e registro de dados;
- Treinos personalizados gerados pela API da OpenAI;
- Resumos e gráficos de evolução;
- Mensagens motivacionais coerentes com o objetivo do usuário.

A aplicação é composta por uma camada Web (WebhookController), um núcleo de negócio (MessageOrchestrator, TelegramService, OpenAiService) e uma camada de persistência baseada em PostgreSQL, onde são armazenados os dados de usuários, registros de peso/medidas e treinos realizados.

3. Fluxo Geral de Interação

De forma resumida, a interação entre o usuário e o FitJourneyAI segue o fluxo abaixo:

1. O usuário envia uma mensagem ao bot pelo Telegram.
2. A Telegram Bot API encaminha um JSON para a aplicação (WebhookController).
3. O MessageOrchestrator analisa o conteúdo e decide qual fluxo deve ser acionado (onboarding, registro, treino, progresso etc.).
4. Quando necessário, o OpenAiService é utilizado para classificar a intenção da mensagem, gerar treinos ou produzir respostas motivacionais.
5. As informações relevantes são persistidas no banco de dados PostgreSQL.
6. O TelegramService envia a resposta formatada de volta para o usuário no Telegram.

A seguir, cada fluxo funcional é detalhado, incluindo os dados envolvidos, a interação com a IA e o tipo de feedback fornecido ao usuário.

4. Fluxo 1 – Onboarding (Cadastro Inicial)

Objetivo: coletar os dados básicos do usuário para personalizar a experiência.

Gatilho: primeira mensagem do usuário ou comando /start.

4.1 Passos do Fluxo

1. Detecção de novo usuário

- O WebhookController recebe a mensagem e repassa para o MessageOrchestrator.
- O sistema verifica no banco de dados se existe um registro associado ao id do Telegram.
- Se não existir, `onboarding\_concluido = false` e o sistema entra no modo de cadastro inicial.

2. Apresentação do bot

- O bot envia uma mensagem de boas-vindas, explicando o propósito do FitJourneyAI.
- Exemplo de mensagem:

"Oi, eu sou o FitJourneyAI, seu parceiro de evolução na academia 🏋️. Para te ajudar de verdade, preciso de algumas informações rápidas sobre você. Podemos começar?"

3. Coleta de dados principais

O bot conduz uma pequena entrevista com perguntas em sequência. Cada resposta é temporariamente armazenada em memória (estado de sessão) até a confirmação final:

- Nome ou apelido: "Como você quer que eu te chame?";
- Objetivo principal: emagrecer, ganhar massa, melhorar desempenho ou manter forma;
- Nível de experiência: iniciante, intermediário ou avançado;
- Frequência semanal de treino: quantos dias por semana pretende treinar;
- Peso atual (obrigatório) e altura (opcional).

4. Confirmação dos dados

- O bot envia um resumo das informações coletadas e pergunta se está tudo correto.
- Em caso de correção, o usuário pode editar algum campo (por exemplo, objetivo ou peso) antes de salvar.
- Em caso de confirmação, os dados são persistidos na tabela `users` e `onboarding\_concluido` é marcado como verdadeiro.

5. Encerramento do onboarding

- O bot envia uma mensagem de conclusão, mostrando os principais comandos disponíveis:

- `/registro` para registrar peso e medidas;
- `/treino` para gerar um treino personalizado;
- `/progresso` para visualizar a evolução;
- `/menu` para ver todas as opções.

• A partir deste ponto, todas as interações passam a utilizar o perfil cadastrado para personalização.

4.2 Dados Persistidos

Na tabela `users` são armazenados, no mínimo:

- id_telegram;
- nome;
- objetivo (enum ou texto categorizado);
- nivel (iniciante, intermediário, avançado);
- frequencia_treino_estimada (dias por semana);
- peso_atual;
- altura (opcional);
- onboarding_concluido (boolean);
- data_cadastro.

5. Fluxo 2 – Menu e Ajuda

Objetivo: apresentar, de forma clara, o que o bot é capaz de fazer e facilitar a navegação.

Gatilhos: comandos /menu ou /ajuda, ou mensagens do tipo "o que você faz?".

5.1 Comportamento do Fluxo

1. O usuário executa o comando /menu ou /ajuda.
2. O MessageOrchestrator identifica o comando e monta uma mensagem com as principais funcionalidades:
 - Registrar peso e medidas – `/registro`;
 - Registrar treinos realizados – `/treino\_feito`;
 - Gerar um treino personalizado – `/treino`;
 - Visualizar progresso e gráficos – `/progresso`;
 - Ver resumo da semana – `/resumo`;
 - Ajustar dados e preferências – `/config`.
3. Sempre que uma mensagem foge completamente do contexto, o fluxo de fallback também direciona o usuário para o `/menu`.

6. Fluxo 3 – Registro de Peso e Medidas

Objetivo: manter um histórico de peso e medidas corporais para avaliação de evolução.

Gatilhos: comando `/registro` ou mensagens que mencionam alteração de peso, como "meu peso hoje é 78,4 kg".

6.1 Registro rápido de peso

1. O usuário informa apenas o peso atual, de forma explícita.
2. O sistema tenta extrair o valor numérico da mensagem (78, 78.4, 78,4 etc.).
3. Caso o valor seja válido, o registro é salvo na tabela `measurements` com os campos: data/hora, peso e demais medidas nulas.
4. O bot responde confirmando o registro e comparando com o último valor disponível, por

exemplo:

"Peso registrado: 78,4 kg. Você está 0,6 kg mais leve do que na última medição!"

6.2 Registro guiado de medidas completas

1. O usuário executa `/registro` ou responde afirmativamente quando o bot pergunta se deseja registrar medidas.
2. O bot inicia uma sequência de perguntas:
 - Peso atual (kg);
 - Cintura (cm);
 - Quadril (cm);
 - Peito (cm);
 - Braço direito e esquerdo (cm);
 - Outras medidas que sejam relevantes para o escopo do projeto.
3. Cada resposta é validada (por exemplo, verificando se é um número positivo plausível).
4. Ao final, o bot apresenta um resumo das medidas e pergunta se deseja confirmar o registro.
5. Após a confirmação, os dados são persistidos na tabela `measurements` associados ao usuário.
6. O bot calcula a variação em relação à última medição e retorna um breve comentário, destacando reduções de medidas em áreas como cintura ou quadril.

6.3 Dados Persistidos

Na tabela `measurements` são armazenados, por exemplo:

- id_usuario;
- data_registro;
- peso;
- cintura;
- quadril;
- peito;
- braco_direito;
- braco_esquerdo;
- outros campos que venham a ser definidos.

7. Fluxo 4 – Registro de Treinos Realizados

Objetivo: acompanhar a consistência e o volume de treino do usuário ao longo do tempo.

Gatilhos: comando `/treino\_feito` ou mensagens como "terminei o treino de pernas".

7.1 Passos do Fluxo

1. O bot pergunta qual grupo muscular foi treinado, oferecendo opções via botões (peito, costas, pernas, ombro, braços, fullbody, cardio etc.).
2. Em seguida, solicita a duração aproximada do treino em minutos.
3. O usuário atribui uma nota de intensidade percebida de 1 a 10.

4. Opcionalmente, o bot pergunta se o usuário deseja registrar os principais exercícios realizados em texto livre.
5. O registro é persistido na tabela `workouts`, contendo: data, grupo muscular, duração, intensidade e descrição livre.
6. O bot envia uma mensagem de confirmação, reforçando o comportamento positivo e lembrando que esse treino será considerado nos resumos semanais e gráficos de progresso.

8. Fluxo 5 – Geração de Treinos Personalizados

Objetivo: fornecer treinos sob demanda, alinhados ao perfil e objetivo do usuário.

Gatilhos: comando `/treino` ou mensagens do tipo "me manda um treino de peito".

8.1 Lógica de Personalização

Para gerar um treino, o backend recupera informações do cadastro e do histórico do usuário, tais como:

- objetivo principal (emagrecimento, hipertrofia, desempenho);
- nível (iniciante, intermediário, avançado);
- frequência semanal de treinos;
- último grupo muscular treinado (para evitar repetições inadequadas);
- existência de registros recentes de treinos ou períodos de inatividade.

Com base nesses dados, é montado um prompt estruturado para a API da OpenAI, especificando:

- Quem é o usuário (perfil resumido);
- Qual grupo muscular ou tipo de treino deve ser gerado (caso já tenha sido pedido pelo usuário);
- Duração aproximada do treino em minutos;
- Limitações importantes (por exemplo, evitar exercícios muito avançados para iniciantes);
- Formato de saída desejado (lista enumerada de exercícios com séries, repetições e tempo de descanso).

8.2 Exemplo simplificado de prompt para treino

Um exemplo de prompt (em português) enviado à OpenAI poderia ser:

"Você é um personal trainer experiente. Gere um treino de musculação focado em pernas para um usuário iniciante cujo objetivo é ganhar massa muscular. O treino deve durar aproximadamente 50 minutos e utilizar apenas equipamentos comuns de academia. Retorne a resposta em português, no formato: nome do exercício, séries x repetições, tempo de descanso, e uma breve dica de execução."

8.3 Resposta e feedback ao usuário

1. A OpenAI retorna um conjunto de exercícios formatados em texto.
2. O backend organiza a resposta, aplicando formatações simples (negrito em nomes de

exercícios, por exemplo) e repassa para o TelegramService.

3. O usuário recebe o treino em forma de lista, podendo executá-lo imediatamente.

4. Opcionalmente, o bot pode perguntar se o usuário deseja registrar esse treino como um "plano de treino" ou simplesmente utilizá-lo como sugestão pontual.

5. Informações básicas sobre o treino sugerido podem ser armazenadas para análise futura, garantindo que o histórico de recomendações também seja considerado nos resumos.

9. Fluxo 6 – Visualização de Progresso e Gráficos

Objetivo: permitir que o usuário acompanhe visualmente sua evolução em termos de peso, medidas e consistência de treinos.

Gatilhos: comando `/progresso` ou mensagens como "me mostra minha evolução".

9.1 Seleção de tipo de progresso

Ao receber o comando, o bot pergunta qual aspecto o usuário deseja visualizar:

- Peso corporal ao longo do tempo;
- Medidas corporais (cintura, quadril, peito etc.);
- Frequência de treinos por semana;
- Visão geral (combinação das opções anteriores).

9.2 Tipos de gráficos sugeridos

Para tornar o acompanhamento mais intuitivo, diferentes tipos de gráficos podem ser utilizados:

- Gráfico de linha para evolução do peso (e eventualmente de medidas específicas) ao longo do tempo;
- Gráfico de barras para quantidade de treinos por semana ou por mês;
- Gráfico de barras agrupadas ou pizza para distribuição de treinos por grupo muscular (peito, costas, pernas etc.).

Os gráficos são gerados no backend (por exemplo, por uma biblioteca de geração de imagens) e enviados ao usuário como imagens PNG pelo Telegram. Dessa forma, o usuário visualiza diretamente no chat uma representação clara de como está sua evolução.

9.3 Resumo textual e análise pela IA

Além das imagens, o sistema envia um resumo textual dos dados, que pode ser complementado por comentários gerados via OpenAI. Por exemplo:

- "Nos últimos 30 dias você perdeu 1,8 kg e reduziu 3 cm de cintura.";
- "Você realizou 14 treinos neste período, com média de 3,5 treinos por semana."

A IA pode ser utilizada para transformar os números em uma análise mais amigável e motivacional, explicando se o ritmo está adequado para o objetivo do usuário e sugerindo pequenos ajustes, se necessário.

10. Fluxo 7 – Resumos Periódicos e Mensagens Motivacionais

Objetivo: fornecer feedback sintético sobre o desempenho recente e reforçar a motivação do usuário.

Gatilho: comando `/resumo` (e, futuramente, rotinas automáticas agendadas).

10.1 Cálculo do resumo

1. O backend consulta os registros de treinos (`workouts`) e de medidas (`measurements`) em um determinado intervalo de tempo (por exemplo, última semana ou último mês).
2. São calculados indicadores como:
 - quantidade de treinos no período;
 - variação de peso em relação ao período anterior;
 - eventuais recordes, como "semana com maior número de treinos";
 - média de intensidade dos treinos (quando esta informação é registrada).

10.2 Mensagens motivacionais com IA

A partir desses indicadores, a API da OpenAI pode ser acionada para produzir uma mensagem motivacional personalizada. O prompt inclui o objetivo do usuário, os números calculados e um pedido de comentário encorajador e realista. Exemplo de instrução:

"Com base nos dados abaixo, escreva uma mensagem motivacional curta e direta para o usuário. Use um tom amigável e focado em disciplina e constância. Não faça promessas irreais. Dados: objetivo = emagrecimento, treinos na semana = 4, variação de peso = -0,4 kg."

A resposta é então enviada ao usuário juntamente com os números brutos, criando uma experiência semelhante à de um treinador comentando os resultados.

11. Fluxo 8 – Ajuda e Tratamento de Mensagens Fora de Contexto

Objetivo: lidar com mensagens que não correspondem a nenhum dos fluxos anteriores, evitando que o usuário fique sem resposta útil.

Gatilhos: comando `/ajuda` ou mensagens que não podem ser mapeadas com segurança para uma funcionalidade específica (intenção desconhecida).

11.1 Detecção de intenção desconhecida

1. Sempre que a mensagem não se encaixa claramente em um comando conhecido ou em um fluxo ativo (por exemplo, durante o onboarding), o MessageOrchestrator pode consultar a OpenAI pedindo uma tentativa de classificação de intenção.
2. Se a IA indicar baixa confiança ou se a mensagem realmente estiver fora do escopo do bot, a intenção é classificada como "desconhecida".

11.2 Resposta de fallback

Quando a intenção é desconhecida, o bot responde com uma mensagem de fallback, explicando de forma leve que não entendeu e sugerindo comandos úteis, por exemplo:

"Não tenho certeza se entendi 😊. Hoje eu consigo te ajudar a registrar dados (/registro), gerar um treino (/treino) ou mostrar sua evolução (/progresso). Use /menu para ver todas as opções."

Dessa forma, o usuário é sempre redirecionado para caminhos bem definidos, mantendo uma boa experiência de uso.

12. Fluxo 9 – Monitoramento de Inatividade e Reengajamento do Usuário

Objetivo: identificar períodos de inatividade do usuário e enviar lembretes ou mensagens motivacionais de forma automática, incentivando o retorno ao uso do bot e a continuidade do acompanhamento da evolução física.

Esse fluxo não é disparado diretamente por um comando do usuário, mas sim por uma rotina agendada no backend, que verifica periodicamente os registros no banco de dados e decide quem deve receber uma mensagem de reengajamento.

12.1 Dados necessários para o monitoramento de inatividade

Para viabilizar o monitoramento de inatividade, alguns campos adicionais são mantidos na tabela de usuários (`users`):

- `last_interaction_at`: data e hora da última interação relevante do usuário com o bot (por exemplo, qualquer mensagem, registro de treino ou de medidas);
- `last_nudge_at`: data e hora do último lembrete de inatividade enviado ao usuário;
- `nudges_enabled`: indicador booleano que determina se o usuário aceita receber lembretes (por exemplo, configurável via comando `/config`).

Sempre que o usuário realiza alguma ação significativa (envia mensagem, registra treino, atualiza peso, etc.), o campo `last_interaction_at` é atualizado. Esse campo serve de base para detectar períodos de inatividade maiores do que o desejado (por exemplo, 5 ou 7 dias sem qualquer interação).

12.2 Rotina agendada (schedule) no backend

No backend desenvolvido em Spring Boot, o monitoramento de inatividade é implementado por meio de uma rotina agendada utilizando a anotação `@Scheduled`. Essa rotina é executada em intervalos regulares (por exemplo, uma vez por hora ou uma vez por dia), realizando os seguintes passos gerais:

1. Determinação do limite de inatividade

- A rotina calcula um instante de tempo de corte, por exemplo: agora menos 5 dias.
- Usuários cuja `last_interaction_at` é anterior a esse limite são considerados inativos para fins de lembrete.

2. Consulta aos usuários inativos

• O repositório de usuários (`UserRepository`) executa uma consulta para buscar todos os usuários que:

- têm `nudges_enabled = true`;
- possuem `last_interaction_at` anterior ao limite de inatividade definido;
- e, opcionalmente, não receberam um lembrete recente (por exemplo, `last_nudge_at` nulo ou também anterior a um determinado intervalo).

3. Geração da mensagem de reengajamento

• Para cada usuário inativo, o sistema gera uma mensagem de lembrete ou motivação, possivelmente utilizando a API da OpenAI para personalizar o conteúdo.

4. Envio da mensagem pelo Telegram

• O `TelegramService` envia a mensagem gerada para o respectivo `chat_id` do usuário.

5. Atualização do registro do usuário

• Após o envio, o campo `last_nudge_at` é atualizado para o instante atual, evitando que o mesmo usuário receba lembretes em excesso.

12.3 Papel da IA no reengajamento

A API da OpenAI vai ser usada para tornar os lembretes de inatividade mais naturais e personalizados. Em vez de enviar apenas uma mensagem fixa, o backend monta um prompt contendo informações como:

- nome do usuário;
- objetivo cadastrado (emagrecer, ganhar massa, desempenho etc.);
- quantidade de dias de inatividade;
- breve descrição do histórico recente (por exemplo, número de treinos realizados na última semana antes de parar).

Um exemplo de instrução para a IA seria:

"Você é um treinador motivacional. O usuário [nome], cujo objetivo é [objetivo], está há [X] dias sem registrar nenhum treino ou medida no sistema. Escreva uma mensagem curta (máximo de 2 linhas), amigável e motivadora, lembrando-o de registrar o peso, atualizar as medidas ou pedir um novo treino. Use um tom positivo e não seja julgador. Responda em português."

A resposta da IA é então enviada diretamente ao usuário pelo Telegram, criando uma sensação de acompanhamento contínuo, semelhante a um treinador atento que percebe períodos de ausência e incentiva o retorno com empatia.

12.4 Experiência do usuário no recebimento do lembrete

Do ponto de vista do usuário, o fluxo de reengajamento funciona de maneira simples: após um período sem enviar mensagens ao bot, ele passa a receber lembretes ocasionais, como, por exemplo:

"Ei, [nome], faz alguns dias que você não registra nada por aqui. Que tal anotar seu peso de

hoje ou me pedir um treino novo? 🏋️ "

Ou, em uma versão gerada pela IA:

"[nome], sua jornada não acabou. Mesmo após alguns dias parado, cada novo registro conta. Me manda seu peso ou pede um treino que eu te ajudo a retomar o ritmo. 🏋️ "

Após receber esse tipo de mensagem, o usuário pode simplesmente responder com comandos como `/registro` ou `/treino`, retomando imediatamente um dos fluxos principais de interação.

13. Considerações Finais

O fluxo de interação do FitJourneyAI foi pensado para ser simples, porém completo, cobrindo desde o primeiro contato do usuário até a visualização de gráficos de progresso e o recebimento de mensagens motivacionais personalizadas. A combinação entre uma arquitetura bem-organizada, persistência de dados estruturada e o uso da API da OpenAI permite que o bot evolua gradualmente, incorporando novos recursos, como perfis de estilos motivacionais e planos de treino mais sofisticados em etapas futuras do projeto.